

POSCO K-뉴딜 아카데미 · 데이터분석 1기

제조 현장의 AI는 어떻게 만들어지는가

포스코DX 현직자 특강 — 비전 AI와 AI 오퍼레이터 개발 현장

현장 데이터	엣지 서버 · 최적화	멀티 에이전트	현업 협의
--------	-------------	---------	-------

2026년 9월 4일 (금) · 포스코DX 현직자 특강

강사 권장훈 대리 (포스코DX 사업실) · 10:00~11:30 · 6개 반 합동 특강

국립금오공과대학교 컴퓨터공학부 박민호

READING GUIDE

이 책을 읽는 법

벋지와 콜아웃을 먼저 익히면 훨씬 빨리 읽힙니다

이 정리는본은 문법서가 아니라 **현장 기록**입니다. 강사가 실제로 겪은 제약과 판단 기준을 항목 단위로 쪼개어 담았습니다. 각 항목은 정의 · 왜 쓰나 · 응용 순서로 읽으면 됩니다.

분류 벋지

벋지	무엇을 가리키나	예
개념	현장에서 통용되는 용어와 사고 틀	정제된 데이터, 암묵지
기술	실제로 쓰이는 도구와 기법	도커, 경량화, 버티컬 LLM
제약	개발을 실제로 묶는 조건	폐쇄망, GPU 슬라이스
구조	시스템이 어떻게 엮여 있는가	멀티 에이전트, PLC 연동
원칙	현장에서 지켜야 하는 행동 기준	개발 범위 협의, 기록
조직	조직이 나뉘는 방식과 역할	연구소, 사업실
역량	신입에게 요구되는 것	회의록, 코드 이해

콜아웃

개념

용어의 뜻과 배경을 한 번 더 짚습니다.

설비 적용

제조 현장에 실제로 어떻게 붙는지를 설명합니다.

주의

놓치면 손해를 보는 조건을 짚습니다.

자주 하는 실수

학생이 흔히 잘못 이해하는 지점을 바로잡습니다.

아카데미 학습과의 연결

지금까지 아카데미에서는 파이썬 문법 · 넘파이 · 판다스로 **정제된 데이터를 다루는 법**을 배웠습니다. 이번 특강은 그 반대편, 즉 **정제되지 않은 현장 데이터**를 다루는 이야기입니다. 배운 것이 쓸모없다는 뜻이 아니라, 배운 것을 쓰기 전에 무엇을 먼저 해결해야 하는지를 알려주는 자리였습니다.

CONTENTS

목차

무엇을 배우는지 먼저 훑어봅니다

장 / STEP	무엇을 배우나
CHAPTER 0 오늘의 지도	특강의 뼈대와 핵심 용어를 먼저 세웁니다
STEP 1 교육 데이터와 현장 데이터	왜 배운 대로 안 되는지, 자유도라는 함정
STEP 2 보안과 비용이라는 두 제약	폐쇄망, 버티컬 LLM, 엣지 서버, GPU 슬라이스, 최적화
STEP 3 연구소와 사업실	POC와 기술 이관, 두 조직의 시선 차이
STEP 4 현장 개발 여섯 가지 원칙	화각 확인부터 시스템 관점까지
STEP 5 AI 오퍼레이터의 구조	멀티 에이전트 오케스트레이션과 PLC 연동
STEP 6 현업을 설득하는 기술	오탐과 미탐, 대안 제시, 룰베이스 논쟁
STEP 7 취업 준비에 대하여	강사의 경로와 신입에게 기대하는 것
부록 A 치트시트	전체 용어를 한 장에 압축
부록 B 오해 · 함정 사전	잘못 알기 쉬운 지점 교정
부록 C 하이라이트와 다음 액션	내 상황에 붙이는 실행 계획

오늘의 지도

특강의 뼈대와 핵심 용어

0.1 이 특강은 무엇에 관한 것인가

강사는 포스코DX 사업실에서 비전 AI와 AI 오퍼레이터를 개발하는 현직 개발자입니다. 2022년 웹 개발 직무로 입사해 현재는 비전과 에이전트 업무를 맡고 있습니다. 특강은 크게 두 덩어리였습니다. 앞쪽은 **본인의 취업 경로**, 뒤쪽은 **제조 현장에서 AI를 개발할 때 실제로 마주치는 문제들**입니다.

강의를 관통하는 문장을 하나로 줄이면 이렇습니다. **모델 성능은 기본이고, 진짜 일은 그 앞뒤에 있다.** 앞은 데이터와 물리 환경, 뒤는 운영과 현업 협의입니다.

0.2 현장 AI 개발의 큰 흐름

개별 항목으로 들어가기 전에 전체 흐름을 잡아 둡니다. 아래 다섯 단계는 특강에서 언급된 순서를 그대로 따릅니다.



학교와 학원에서 배우는 것은 대개 세 번째 칸 하나입니다. 나머지 네 칸이 현업의 실제 업무 시간을 대부분 잡아먹습니다. 이 정리본은 그 네 칸에 무게를 둡니다.

0.3 핵심 용어 미리보기

본문에서 자세히 다루기 전에 이름만 먼저 눈에 익혀 둡니다. 특강에서 별도 설명 없이 지나간 용어도 포함했습니다.

용어	한 줄 뜻	어디서 나오나
온프레미스 · 폐쇄망	데이터를 외부로 내보내지 않고 사내에서만 처리	STEP 2
버티컬 LLM	특정 도메인에 맞게 파인튜닝한 자체 언어모델	STEP 2
엣지 서버	현장 가까이 두는 저사양 · 저비용 추론 서버	STEP 2
GPU 슬라이스	한 GPU를 나눠 쓰도록 자원을 할당하는 방식	STEP 2
POC	본 프로젝트 전에 가능성만 확인하는 시범 검증	STEP 3

용어	한 줄 뜻	어디서 나오나
암묵지	문서화되지 않은 채 작업자 머릿속에만 있는 지식	STEP 4
도커	환경까지 통째로 담아 옮기는 컨테이너 기술	STEP 4
하네스 엔지니어링	에이전트가 엉뚱하게 굴지 않도록 가드를 치는 작업	STEP 5
레이턴시	신호를 보낸 뒤 설비가 실제로 움직일 때까지의 지연	STEP 5
오탐 · 미탐	없는데 잡는 것 · 있는데 놓치는 것	STEP 6
RAG	내부 문서를 검색해 답변 근거로 붙이는 기법	STEP 7

STEP 1

교육 데이터와 현장 데이터

왜 배운 대로 되지 않는가

강사가 데이터분석 수강생을 앞에 두고 가장 먼저 꺼낸 이야기입니다. 배운 것이 틀렸다는 뜻이 아니라, 조건이 다르다는 뜻입니다.

개념 정제된 데이터 교육용 데이터의 성질

정의 분석 결과가 잘 보이도록 미리 손질된 데이터입니다. 이상치가 의도적으로 몇 개 섞여 있고, 그 이상치를 처리하면 결과가 눈에 띄게 좋아지도록 설계되어 있습니다.

왜 쓰나 교육은 시간이 부족합니다. 정해진 시간 안에 **분석 기법 자체**를 익히게 하려면 데이터가 앞을 가로막으면 안 됩니다. 그래서 데이터를 보는 순간 어떤 흐름으로 풀어 결론을 내면 되는지가 얼핏 보입니다.

응용 아카데미 실습에서 쓴 `10_mct_tool.csv` 같은 파일을 떠올려 보면 컬럼명이 정돈되어 있고 결측이 관리되어 있습니다. 현장 데이터를 받으면 그 상태를 **내가 직접 만들어야** 합니다.

개념

정제된 데이터로 연습하는 것은 필요한 과정입니다. 다만 그 단계가 실무의 전부가 아니라 **가장 마지막 단계**라는 점을 알고 있어야 합니다.

제약 데이터의 자유도 현장 데이터가 어려운 진짜 이유

정의 쓸 수 있는 데이터가 지나치게 많아, 무엇을 써야 할지부터 스스로 정해야 하는 상태입니다. 강사의 표현으로는 자유가 너무 많은 상황입니다.

왜 쓰나 선택지가 많다는 것은 정답이 없다는 뜻입니다. 어떤 컬럼을 쓸지, 어느 기간을 자를지, 어떤 설비를 묶을지가 전부 판단의 대상이 됩니다. 이 판단이 분석의 절반을 차지합니다.

응용 현장 데이터를 반출하는 절차 자체가 까다롭기 때문에, 실무에서는 **한 번 가져올 때 최대한 많이** 가져옵니다. 무엇이 필요할지 미리 알 수 없으니 넓게 받아 두고 그 안에서 골라내는 방식입니다.

자주 하는 실수

가장 흔한 실패는 결론을 먼저 정해 놓고 **데이터를 거기에 끼워 맞추는** 것입니다. 강사가 현장에서 자주 본다고 직접 언급한 패턴입니다. 결론이 먼저 서면 반대 증거를 찾지 않게 됩니다.

원칙 모델보다 데이터 먼저 작업 순서

정의 모델을 만들기 전에 현장 데이터가 어떤 상태인지 직접 확인하는 것을 우선합니다.

왜 쓰나 모델은 데이터를 넘어설 수 없습니다. 수집 주기가 성기거나 라벨이 없으면 어떤 알고리즘을 써도

결과가 나오지 않습니다. 강화학습을 하는 동료들도 **데이터를 넓게 받아 분석한 뒤에야** 어떤 데이터로 모델을 만들지 정한다고 합니다.

응용 예지보전 주제라면 순서가 이렇게 됩니다. 센서 태그 목록 확보 → 수집 주기 확인 → 정비 이력과 시간 정합 → 라벨 생성 가능 여부 판단 → 그 다음에 모델.

설비 적용

설비 진동으로 베어링 결함을 잡으려면 kHz 단위 샘플링이 필요합니다. 수집 주기가 1초라면 모델을 바꾸는 것이 아니라 **센서와 수집 설정을 바꾸는** 일부터 해야 합니다.

STEP 2

보안과 비용이라는 두 제약

현장 개발을 실제로 묶는 조건

특강에서 가장 많은 시간을 쓴 대목입니다. 강사가 입사한 2022년과 지금이 크게 달라진 부분이기도 합니다.

제약 온프레미스 · 폐쇄망 데이터는 밖으로 나갈 수 없다

정의 외부 인터넷과 분리된 사내 환경에서만 데이터를 처리하는 구조입니다. 제철소 데이터는 외부 반출이 금지되어 있습니다.

왜 쓰나 공정 데이터는 그 자체가 기술 자산입니다. 밖으로 나가면 안 되기 때문에 상용 API를 그대로 호출하는 방식은 **선택지에서 아예 빠집니다**. 좋은 모델이 있어도 데이터를 보낼 수 없으면 쓸 수 없습니다.

응용 그래서 자체 LLM을 사내에 올려 돌립니다. 외부 서비스를 부르는 대신 모델을 안으로 들여오는 방향입니다.

설비 적용

현장에 서버를 설치하면 인터넷이 되지 않습니다. `pip install` 로 라이브러리를 받거나 허깅페이스에서 모델을 내려받는 일이 **불가능합니다**. 이 제약이 뒤에 나올 도커 사용의 직접적인 이유가 됩니다.

기술 버티컬 LLM 도메인 특화 자체 모델

정의 오픈소스 언어모델을 가져와 자사 공정에 맞게 파인튜닝한 모델입니다. 특정 수직 도메인에 맞춘다는 뜻에서 버티컬이라 부릅니다.

왜 쓰나 유료 라이선스 모델은 쓰기 어렵습니다. 프로젝트에 들어가는 순간 그 **라이선스 비용이 현업에 청구** 되기 때문입니다. 강사의 표현으로는 현업이 당연히 난색을 보입니다. 반면 성능이 좋은 오픈소스 모델은 이미 충분히 많습니다.

응용 파인튜닝은 연구소에서 수행하고, 사업실은 그 결과를 받아 현장에 적용합니다. 이 분업 구조는 STEP 3에서 다시 다룹니다.

개념

오픈소스를 쓰는 이유가 기술적 선호가 아니라 **비용과 보안** 때문이라는 점이 중요합니다. 취업 준비 관점에서는 상용 API 경험만이 아니라 오픈소스 모델을 직접 띄워 본 경험이 의미를 갖습니다.

기술 엣지 서버 현장 가까이 두는 저사양 추론기

정의 고성능 중앙 서버 대신 현장 근처에 두는 상대적으로 가벼운 서버입니다. 성능은 낮지만 비용이 싸고 빠르게 도입할 수 있습니다.

왜 쓰나 GPU와 램 가격이 크게 올랐고, 서버를 주문하면 **두세 달이 걸립니다**. 아무리 좋은 프로젝트라도 비용이 수십억으로 넘어가면 현업이 시도 자체를 못 합니다. 비전처럼 큰 모델이 필요 없는 작업은 엣지로 내려서 비용과 리드타임을 줄입니다.

응용 프로젝트 성격에 따라 갈립니다. 순수 비전은 엣지 서버, LLM이 섞인 AI 오퍼레이터는 고사양 서버를 씁니다. 어느 쪽으로 갈지 개발자가 **의견을 내고 PM이 결정**합니다.

설비 적용

전기실 화재 감지처럼 현장에서 즉시 판단해야 하는 작업은 엣지가 유리합니다. 데이터를 밖으로 보내지 않아도 되고 지연도 줄어듭니다.

제약 GPU 슬라이스 자원 할당 상한

정의 하나의 물리 GPU를 여러 가상 서버로 나누어, CPU · GPU · 램을 정해진 몫만큼만 쓰도록 제한하는 방식입니다.

왜 쓰나 전용 서버를 통째로 받던 시절에는 최적화를 안 해도 돌았습니다. 지금은 할당받은 몫 안에서 돌려야 하므로 **최적화가 성능이 아니라 가능 여부를 결정**합니다.

응용 강사가 든 예를 그대로 계산해 보면 규모의 차이가 분명해집니다. 전용 서버 한 대로 CCTV 15대를 추론하던 것이, 슬라이스 환경에서는 한 몫당 네 대 수준으로 떨어집니다. 대신 같은 사양을 여러 개 할당받아 합산하는 방식으로 규모를 맞춥니다.

```
total_cam = 15      # 전용 서버 1대로 처리하던 카메라 수
slice_cam = 4        # 슬라이스 1개로 처리 가능한 수
slices     = 7        # 할당받은 슬라이스 개수

print(slice_cam * slices)
-----
> 28
```

주의

최적화가 부족하면 당장은 돌아가도 부하가 누적됩니다. 몇 달, 길게는 1~2년 돌리다 **서버가 갑자기 죽고** 그 대응까지 개발자 몫이 됩니다. 여유 사양에서는 드러나지 않던 문제가 슬라이스 환경에서는 반드시 드러납니다.

기술 경량화와 추론 최적화 제한된 자원에서 성능을 내는 법

정의 모델을 그대로 쓰지 않고 크기를 줄이거나 추론 엔진을 바꿔 같은 자원에서 더 많이 처리하도록 만드는 작업입니다.

왜 쓰나 노후 서버를 그대로 써야 하는 상황이 실제로 생깁니다. 그럴 때 프로젝트를 성사시키는 방법은 서

버를 바꾸는 것이 아니라 **모델 쪽을 줄이는** 것입니다. 두세 개밖에 못 돌리던 것을 다섯, 여섯 개까지 올리는 것이 목표가 됩니다.

응용 특강에서 언급된 방법은 두 가지입니다. 하나는 모델 경량화, 다른 하나는 TensorRT 같은 추론 엔진으로 변환하는 것입니다. 운영 단계에서는 Triton 기반으로 실시간 추론을 구성했다고 했습니다.

개념

성능과 최적화는 대립 관계가 아닙니다. **자원이 제한되면 최적화가 곧 성능**입니다. 강사가 성능보다 최적화가 더 중요할 수도 있다고 말한 배경이 여기에 있습니다.

STEP 3

연구소와 사업실

같은 회사, 다른 시선

진로를 정할 때 직접적으로 쓰이는 구분입니다. 강사는 사업실 소속이며, 두 조직의 시선이 살짝 다르다고 표현했습니다.

조직 연구소 / 사업실 역할 분담 구조

정의 연구소는 기술의 가능성을 먼저 확보하고, 사업실은 그 기술을 이관받아 현장에 적용하고 확산해 수익을 냅니다.

왜 쓰나 검증되지 않은 기술을 곧바로 현장에 넣을 수 없기 때문입니다. 연구소가 먼저 시행착오를 겪어 두면 사업실은 그만큼 시간을 아깁니다. 강사는 연구소가 **시행착오를 줄여 주는 역할**을 한다고 정리했습니다.

응용 센서 퓨전이 이 흐름을 그대로 보여 줍니다. 연구소가 라이다를 먼저 도입해 기술을 확보했고, 그 기술이 지금 사업실로 내려오는 중입니다.

개념

진로 선택에 그대로 대응됩니다. 기술을 파고드는 쪽이 맞으면 연구소, 내가 만든 것이 현장에서 실제로 도는 것을 두 눈으로 보고 싶으면 사업실입니다. 둘 중 어느 쪽이 우월한 것이 아니라 **성향의 문제**입니다.

개념 POC 가능성만 확인하는 시범 검증

정의 본 프로젝트 이전에 이 방식으로 하면 되겠다는 가능성을 확인하는 단계입니다. 안정화된 상태를 만드는 단계가 아닙니다.

왜 쓰나 전체 투자를 결정하기 전에 위험을 줄이기 위해서입니다. 영업에서 들어온 건이 임원을 거쳐 PM 선에서 한 번 걸러지고, 그중 해 볼 만한 것이 POC로 내려옵니다.

응용 POC 단계에서는 현장에 카메라가 아직 없는 경우가 많습니다. 그래서 현업에 요청해 카메라를 지원받거나, 연구소 카메라를 **반입 신청**해 들고 들어가 직접 촬영합니다.

주의

POC 결과는 대체로 긍정적으로 나옵니다. 내가 원하는 화각과 원하는 조건에서 찍은 데이터로 검증하기 때문입니다. **POC 성공이 곧 현장 성공이 아니라는 점**을 전제로 두어야 합니다.

자주 하는 실수

POC가 되면 다 되는 것으로 이해하기 쉽습니다. 실제로는 그때는 됐는데 지금은 안 되거나, 그 자리에 설치 자체가 불가능한 경우가 나옵니다. 다만 그럴 때 못 한다고 끝내지 않고 **풀어 나갈 대안**을 함께 내는 것이 일하는 방식입니다.

원칙 기술 이관 시 질문법 받을 때 무엇을 묻는가

정의 연구소에서 모델을 넘겨받을 때 결과물만 받지 않고, 그 선택의 이유까지 함께 묻는 방식입니다.

왜 쓰나 연구소 모델을 그대로 현장에 넣으면 대개 동작하지 않습니다. 물리적 환경이 다르기 때문입니다.

이유를 알아야 현장 조건에 맞게 **다시 조정할 수 있습니다.**

응용 강사가 실제로 던지는 질문은 세 가지 유형입니다. 이 기술을 그대로 적용하면 되는가, 왜 이 방식을 택했는가, 다른 모델은 무엇을 시도해 보았고 왜 이것이 가장 잘 나왔는가.

설비 적용

이 질문법은 학생 신분에서도 그대로 쓸 수 있습니다. 남의 코드를 참고할 때 동작만 확인하지 말고 **왜 그 선택인지**를 묻는 습관이 그대로 실무 역량이 됩니다.

STEP 4

현장 개발 여섯 가지 원칙

강사가 직접 꼽은 항목들

강사가 개발하면서 말하고 싶은 것을 여섯 가지로 추려 왔다고 밝힌 부분입니다. 순서까지 강의 그대로 옮겼습니다.

원칙 1. 카메라와 화각을 직접 확인한다 개발 전에 현장에 간다

정의 설치된 카메라를 쓸 수 있는지, 원하는 화각이 나오는지를 개발 전에 직접 눈으로 확인하는 것입니다.

왜 쓰나 카메라가 있다고 쓸 수 있는 것이 아닙니다. 아날로그 카메라이거나, 화질이 떨어지거나, 각도가 비스듬하거나 너무 낮게 달려 있으면 **성능이 나올 수 없습니다.**

응용 원하는 위치에 새로 달면 되지 않느냐고 생각하기 쉽지만, 카메라 하나를 놓으려면 전선 포설부터 여러 프로세스를 거쳐야 합니다. 결국 원하지 않는 화각으로 성능을 내야 하는 상황이 자주 생깁니다.

주의

이때 반드시 해야 하는 것은 **우려를 미리 말하는 것**입니다. 할 수 있을 것 같다고만 해 놓고 설치 후에 안 된다고 하면 되돌릴 수 없습니다. 이 화각이면 이런 우려가 있다는 말을 먼저 남겨 두어야 합니다.

현장 2. 현장 인터뷰로 암묵지를 캔다 작업자에게 직접 묻는다

정의 문서로 정리되지 않고 작업자 머릿속에만 있는 판단 기준을 인터뷰로 끌어내는 작업입니다.

왜 쓰나 현업 PM도 대략만 알지 실제 작업자의 판단까지는 모릅니다. 물체를 어느 위치에서 자를지 같은 기준이 사람마다 다르고, 각자 수년에서 수십 년의 노하우로 그렇게 하고 있습니다. **누구도 정리해 놓지 않은 지식**입니다.

응용 인터뷰를 해 보면 사람마다 답이 다르고 그런데 어느 쪽도 문제가 없습니다. 강사가 내린 결론은 모든 사람의 말에 휘둘리지 말고, 인터뷰로 모은 뒤 **우리 나름의 표준을 정하자**는 것이었습니다.

개념

그래서 AI 오퍼레이터를 설명할 때 이런 표현을 씁니다. 암묵지로 흩어져 있던 것을 자동화함으로써 **그것이 표준이자 가이드가 된다.** 자동화의 부수 효과가 아니라 설득 논리 자체입니다.

자주 하는 실수

현업이 준 자료만으로 몇 달 개발한 뒤 현장 확인 없이 적용하면, 작업자들이 왜 이렇게 작동하냐며 **아예 쓰지 않을 확률이 매우 큽니다.** 만드는 것과 쓰이게 하는 것은 다른 문제입니다.

원칙 3. 개발 범위를 보수적으로 협의한다 할 수 있다고 먼저 말하지 않는다

정의 현업의 요구를 받을 때 무엇을 하고 무엇을 미룰지 초반에 명확히 나누는 작업입니다.

왜 쓰나 프로젝트는 중간에 요구가 추가되는 경우가 매우 많습니다. 어느 정도 됐다 싶을 때 이것도 되냐는 요청이 들어옵니다. 초반에 경계를 긋지 않으면 집중해야 할 개발보다 **결가지에 시간을 뺏깁니다.**

응용 강사가 선호하는 PM 유형은 의지가 확고한 쪽입니다. 될 것과 안 될 것을 분명히 갈라 주고, 추가 요청이 오면 지금 것을 끝내고 고도화로 넘기자고 정리해 주기 때문입니다.

주의

혼자 알아 놓으면 한 달 안에 가능하다는 판단이 서도, 그것은 **리스크가 큰 판단**입니다. 중간에 막히면 타 부서 협의를 필요하고 그 절차만 며칠에서 몇 주가 걸립니다. 기한 내에 문제없이 가능한가까지 계산해야 합니다.

자주 하는 실수

신입이 현업과 직접 협의할 기회는 많지 않습니다. 그러니 **PM과의 소통에 집중**하는 것이 맞습니다. 의견을 내지 않는 것이 아니라, 내는 통로가 PM이라는 뜻입니다.

원칙 4. 현장 데이터를 먼저 본다 모델 개발 이전 단계

정의 모델을 만들기 전에 실제 현장 데이터가 어떤 상태인지 확인하는 것입니다. STEP 1에서 다룬 내용과 이어집니다.

왜 쓰나 현장 데이터를 사내로 반출하는 절차 자체가 까다롭습니다. 여러 번 요청하기 어려우므로 **한 번에 넓게 받아** 분석한 뒤 어떤 데이터로 모델을 만들지 정합니다.

응용 강화학습처럼 결과가 눈에 바로 보이지 않는 분야일수록 이 사전 분석의 비중이 큼니다. 비전은 못 잡으면 바로 보이지만, 강화학습은 분석과 검증에 훨씬 많은 시간을 씁니다.

역량 5. 질문하고 기록한다 신입에게 가장 크게 기대하는 것

정의 회의 내용을 정리해 남기고, 이해하지 못한 부분을 질문으로 만드는 습관입니다.

왜 쓰나 신입은 회의에 계속 불려 갑니다. 처음에는 무슨 말인지 이해하지 못한 채 듣게 되는데, PM은 그 상태에서 **회의록을 요구**합니다. 무엇이 궁금한지, 무엇을 이해 못 했는지를 묻습니다. 도메인 지식에 빨리 녹아들게 하려는 목적입니다.

응용 더 근본적인 이유는 기록 자체가 기본이기 때문입니다. 같은 회의를 하고도 현업과 우리의 생각이 같아질 수는 없습니다. 애매한 상황에서 어떻게 개발하기로 했는지를 명확히 남겨 두어야 **나중에 말이 달라져도 방향을 지킬 수 있습니다.**

개념

강사는 아래 신입이 회의록을 잘 써 준 것만으로 크게 만족했다고 말했습니다. 신입에게 많은 것을 바라지 않는다는 뜻이자, **기록이 그만큼 실질적인 기여**라는 뜻이기도 합니다.

설비 적용

팀 프로젝트에서도 똑같습니다. 나는 A를 말했는데 팀원은 B로 이해하고 있는 일이 늘 벌어집니다. 말이 아니라 글로 남겨야 오해가 줄어듭니다.

원칙 6. 시스템 관점으로 본다 개발이 끝이 아니다

정의 모델이 도는 것에서 멈추지 않고 운영 단계까지 고려해 설계하는 관점입니다.

왜 쓰나 개발이 끝나고 잘 돌아간다고 끝이 아닙니다. 서버를 어디에 둘지, 막힌 구간은 없는지, 운영 환경은 무엇으로 할지가 남습니다. 연구소와 사업실의 시선 차이가 **바로 이 지점**입니다.

응용 운영은 원래 윈도우를 많이 썼습니다. 눈으로 보기 편하고 관리가 쉽기 때문입니다. 지금은 리눅스로 옮겨 가는 중입니다.

설비 적용

모델 성능 지표만 있는 포트폴리오와, 어디에 배포해 어떻게 운영할지까지 적힌 포트폴리오는 읽히는 무게가 다릅니다.

기술 도커 환경을 통째로 옮기는 방법

정의 실행에 필요한 라이브러리와 환경을 컨테이너 하나에 담아, 어느 서버에서든 같은 상태로 띄우는 기술입니다.

왜 쓰나 현장 서버는 인터넷이 되지 않습니다. 라이브러리를 그때그때 내려받아 환경을 맞추는 일이 **불가능**합니다. 환경을 통째로 맡아서 가져가면 현장에서 시행착오를 겪지 않아도 됩니다.

응용 판교에서 개발한 환경 그대로 현장 서버에 컨테이너를 띄우면 동일하게 동작합니다. 환경의 제약이 줄어드는 것이 핵심 이득입니다. 더 나아가면 슈퍼바이저로 서버가 죽었다 살아났을 때 자동으로 다시 띄우는 구성까지 갑니다.

주의

강사는 도커를 능숙하게 다룰 필요는 없다고 했습니다. 본인도 입사 후에 배웠습니다. **리눅스를 써 봤고 도커를 띄워 봤다** 정도면 비전 쪽 기본기로 충분하다는 것이 강사의 기준입니다. 형상 관리는 깃을 기본으로 씁니다.

4.7 여섯 원칙 한눈에 보기

여섯 항목은 서로 독립적인 조언이 아니라 **하나의 시간 축** 위에 놓여 있습니다. 개발 전에 해야 할 일, 개발 중에 지켜야 할 일, 개발 후에 남는 일로 나누어 보면 순서가 분명해집니다.

시점	원칙	실패하면 생기는 일
개발 전	1. 카메라와 화각 확인	설치 후에야 성능이 안 나오는 것을 알게 됩니다
개발 전	2. 현장 인터뷰로 암묵지 수집	완성해도 작업자가 쓰지 않습니다
개발 전	4. 현장 데이터 먼저 확인	모델을 다 만든 뒤 데이터가 부족함을 알게 됩니다
개발 중	3. 개발 범위 보수적 협의	요구가 계속 붙어 본 개발이 밀립니다
개발 중	5. 질문과 기록	합의 내용이 사라져 방향이 흔들립니다
개발 후	6. 시스템 관점	돌아가지만 운영할 수 없는 결과물이 남습니다

눈여겨볼 점은 여섯 중 셋이 **개발 전 단계**에 몰려 있다는 것입니다. 코드를 쓰기 전에 확인해야 할 것이 그만큼 많다는 뜻이고, 학교 과제와 실무가 가장 크게 갈리는 지점이기도 합니다.

연결

STEP 5에서는 이 원칙들이 실제 프로젝트에 어떻게 적용되는지를 **AI 오퍼레이터** 사례로 확인합니다. 특히 3번 개발 범위 협의와 6번 시스템 관점이 어떻게 구조 설계로 이어지는지 보게 됩니다.

STEP 5

AI 오퍼레이터의 구조

멀티 에이전트와 설비 제어

강사가 현재 개발 중인 프로젝트입니다. 비밀유지 계약 때문에 구체적인 적용처는 밝히지 않고 구조와 기술만 공유했습니다.

전체 형태는 트리 구조입니다. 메인 에이전트 아래에 서브 에이전트가 있고, 각 서브 에이전트가 자기 툴을 호출합니다.



구조 멀티 에이전트 오케스트레이션 일을 나눠 시키는 구조

정의 메인 에이전트가 트리거를 감지해 적합한 서브 에이전트에 일을 배분하고, 서브 에이전트는 스스로 판단해 자기가 쓸 수 있는 툴을 골라 실행하는 구조입니다.

왜 쓰나 단일 LLM으로는 한계가 있기 때문입니다. 하나의 모델에 모든 판단을 맡기는 대신 역할을 쪼개면, 각 단계에서 무엇이 잘못됐는지 확인하기도 쉬워집니다.

응용 구현에는 LangChain과 LangGraph를 씁니다. 프로젝트 자체에는 최적화와 강화학습도 포함되어 있지만 특강에서는 제외하고 소개했습니다.

개념

특강에서 이름이 명시된 서브 에이전트는 **비전 서브 에이전트** 하나입니다. 나머지 구성은 밝히지 않았으므로, 트리 구조라는 형태만 사실로 받아들이면 됩니다.

기술 하네스 엔지니어링 에이전트에 가드를 치는 일

정의 에이전트가 지시한 대로만 동작하도록 사전에 규칙과 안전장치를 촘촘히 걸어 두는 작업입니다.

왜 쓰나 시킨 대로 항상 잘 동작하지 않기 때문입니다. 이렇게 하라고 했는데 갑자기 다른 일을 하고 있을 수 있습니다. 설비를 다루는 환경에서 그런 이탈은 허용되지 않습니다.

응용 가드를 많이 치면 거의 **롤베이스처럼 보이기도 합니다**. 이 상황에서는 무조건 이렇게 하라는 식으로 계속 인지시키는 방식이기 때문입니다.

개념

중요한 것은 이 울타리가 영구적이지 않다는 점입니다. LLM과 VLM 성능이 올라가면 울타리를 하나씩 걷어낼 수 있고, 그러면 에이전트가 스스로 툴을 더 자유롭게 갖다 쓰게 됩니다. 그 끝에 **자기 진화 에이전트**가 있습니다.

설비 적용

지금 룰베이스처럼 보이더라도 에이전트 구조로 시작해야 하는 이유가 여기 있습니다. 나중에 하면 되지 않느냐는 말에 대한 답은, 지금부터 준비해야 그 단계를 밟아 나갈 수 있다는 것입니다.

구조 PLC 연동과 레이턴시 마지막 한 흡이 물리 세계

정의 에이전트의 판단을 실제 설비 동작으로 옮기는 구간이며, 신호를 보낸 뒤 설비가 움직일 때까지의 지연이 핵심 문제가 됩니다.

왜 쓰나 처음에는 사내 데이터센터에 서버를 두고 PLC와 통신하려 했습니다. 그런데 자르라는 신호를 보내면 **1초 뒤에 움직였습니다**. 설비 제어에서 1초는 치명적입니다.

응용 해법은 네트워크를 최대한 타지 않게 하는 것입니다. 공장 안에 직접 구현해 다이렉트로 연결하면 지연을 크게 줄일 수 있습니다.

설비 적용

소프트웨어만 아는 사람이 막히는 지점이 정확히 여기입니다. 임베디드와 PLC를 함께 아는 사람이 희소한 이유이기도 합니다.

주의

PLC를 건드리는 일 자체가 매우 조심스럽습니다. 설비와 직접 연결되어 있어 문제가 생기면 **설비가 전부 멈출 수 있습니다**. 그래서 추가 요청은 최대한 하지 않고, PLC에 이미 있는 것을 그대로 가져다 쓰려고 합니다.

제약 레거시 설비와 서버 교체 주기 5년마다 판단한다

정의 기존 서버를 그대로 쓸지 교체할지를 5년 주기로 판단하고, 교체 시 뒤따르는 환경 이전 작업을 수행하는 일입니다.

왜 쓰나 서버를 바꾸면 GPU가 올라가고 CUDA 버전이 달라집니다. 그러면 기존에 돌던 것들의 **마이그레이션 작업**이 함께 필요해집니다.

응용 디지털 트윈도 이 맥락에서 언급됐습니다. 언리얼이나 아이작 심으로 구현하지만 시작이 수십억 단위이고, 가상 화면에서 설비를 실제로 제어하는 데까지는 리스크가 커서 그 부분을 빼기도 합니다. 그러면 현업은 실제 설비와 똑같이 움직이는 것을 보기만 하게 되어 잘 쓰지 않게 됩니다.

개념

강사의 평가는 디지털 트윈이 **가격 대비 효과가 아직 크지 않다**는 것입니다. 관련 인력도 팀이라 부르기 어려울 만큼 적습니다. 다만 차근차근 나아가고 있는 영역입니다.

5.5 이 구조에서 신입이 맡게 되는 부분

구조를 이해했다면 다음 질문은 이것입니다. **신입이 이 안에서 무엇을 하게 되는가**. 특강 내용을 종합하면 아래처럼 정리됩니다.

계층	주로 다루는 일	신입 참여도
메인 에이전트	흐름 설계, 라우팅 규칙, 상태 관리	경력자 · PM 주도
서브 에이전트	도메인별 판단 로직, 프롬프트와 가드 설계	일부 참여
툴	모델 추론 호출, 데이터 조회, 신호 읽기	신입이 맡기 좋은 구간
PLC 연동	통신 구성, 지연 최소화, 안전 확인	설비 리스크로 신중 접근

가장 아래 툴 계층이 진입점입니다. 강사도 웹 개발로 들어와 주변 업무를 하면서 비전으로 넘어갔습니다. **눈에 보이는 작은 구간부터 확실히 맡는 것이 현실적인 출발점**입니다.

연결

여기까지가 만드는 이야기였습니다. STEP 6에서는 만든 것을 **현업이 받아들이게 하는 이야기**로 넘어갑니다. 기술보다 대화가 중심이 되는 영역입니다.

STEP 6

현업을 설득하는 기술

안 된다고 끝내지 않는 법

AI 업무를 하면 설득을 굉장히 많이 하게 된다고 강사가 강조한 부분입니다. 기술 문제가 아니라 대화의 문제입니다.

지표 오탐과 미탐 두 오류를 다르게 다룬다

정의 오탐은 없는 것을 있다고 잡는 오류이고, 미탐은 있는 것을 놓치는 오류입니다. 안전 관제에서는 이 둘의 무게가 전혀 다릅니다.

왜 쓰나 강사의 기준은 명확합니다. **안전에서 미탐은 있어서는 안 됩니다.** 반면 오탐은 어느 정도 허용됩니다. 오탐은 사후 분석이 가능하기 때문입니다. 모델 성능이 떨어졌는지, 어떤 유사 상황에서 잘못 잡았는지를 확인해 고쳐 나갈 수 있습니다.

응용 실제로 계산해 보면 두 오류가 서로 다른 지표에 붙는 것이 보입니다. 미탐은 재현율을, 오탐은 정밀도를 떨어뜨립니다.

```
actual = [1,1,1,1,1,1,1,1,0,0,0,0,0,0,0,0,0,0]
predict = [1,1,1,1,1,1,0,0,1,1,1,0,0,0,0,0,0,0]

TP = sum(1 for a,p in zip(actual,predict) if a==1 and p==1)
FN = sum(1 for a,p in zip(actual,predict) if a==1 and p==0) # 미탐
FP = sum(1 for a,p in zip(actual,predict) if a==0 and p==1) # 오탐

print(TP, FN, FP)
print(round(TP/(TP+FN),3)) # 재현율
print(round(TP/(TP+FP),3)) # 정밀도

-----
> 6 2 3
> 0.75
> 0.667
```

설비 적용

예지보전도 같은 구조입니다. 고장을 놓치면 설비가 서지만, 잘못 알람이 울리면 정비 인력이 헛걸음합니다. 어느 쪽을 더 싫어하는지는 **현업과 반드시 합의해야 하는 항목**입니다.

주의

위험 요소의 우선순위는 개발자가 정하는 것이 아닙니다. 무조건 현업과 소통해 정합니다. 다만 현업이 이 부분에 신경을 많이 쓰는 쪽이면 소통이 잘 되고, 아니면 **우리가 찾아가서 말해야 하는 경우도 많습니다.**

원칙 대안을 붙여 말한다 설득의 기본형

정의 불가능한 요구를 받았을 때 안 된다고만 하지 않고, 되는 조건과 안 될 때의 대응까지 함께 제시하는 화법입니다.

왜 쓰나 요즘 현업은 100퍼센트에 가까운 성능을 요구합니다. 카메라만으로는 100퍼센트가 나오지 않습니다. 사람도 눈으로 보다가 놓칠 수 있는 것이 카메라입니다. 그런데 무조건 안 된다고 답하면 현업 입장에서는 **그럼 이 프로젝트를 할 필요가 없다**가 되어 버립니다.

응용 그래서 이런 형태로 말합니다. 이런 경우에는 무조건 가능합니다. 다만 날씨가 나쁘거나 빛 반사가 심할 때는 오인식이 생길 수 있습니다. 오인식이 발생하면 다른 프로세스로 이렇게 대응하겠습니다.

개념

강사가 짚은 핵심은 현업도 말이 안 통하는 사람이 아니라는 것입니다. **그분들도 뒤편에 보고할 거리가 필요할 뿐입니다.** 왜 안 되는지와 안 될 때 어떻게 할지를 주면 그것으로 보고가 됩니다.

개념 룰베이스 대 에이전트 가장 자주 받는 반문

정의 트리거를 받아 정해진 동작을 하는 것이면 규칙 기반으로 짜면 되지 않느냐는 현업의 문제 제기와, 그에 대한 답변 구조입니다.

왜 쓰나 룰베이스가 개발하기 편하고 비용도 쌉니다. 무엇이 들어오면 무엇을 하고, 예외는 이렇게 처리한다는 식으로 짜면 됩니다. 그래서 현업은 당연히 이 질문을 던집니다.

응용 답변의 축은 **예상하지 못한 상황**입니다. LLM과 VLM을 쓰면 우리가 구현해 두지 않은 상황에서도 스스로 판단할 수 있습니다. 지금 당장은 비슷해 보여도 점차 이쪽이 필요해진다는 논리로 설득합니다.

주의

현업도 스스로 생성형 AI에 물어보고 지식을 쌓습니다. 그래서 이걸 당연한 건데 왜 안 되냐는 식의 질문이 **많아졌습니다.** 이럴 때는 그것이 최종적인 모습이고 지금은 어느 단계에 있는지를 정확히 설명하는 것이 답입니다.

제약 실외 환경의 불확실성 비전이 어려워지는 조건

정의 공장 외부에 설치된 카메라를 쓸 때 발생하는 통제 불가능한 변수들입니다.

왜 쓰나 먼지가 렌즈에 붙고, 밤이 되면 조명이 바뀌고, 빛 반사가 심해집니다. 그리고 이 조건은 **카메라마다**

장소마다 전부 다릅니다.

응용 화질이 좋고 날씨가 좋으면 거의 100퍼센트에 가까운 성능이 나옵니다. 문제는 조건이 조금만 나빠져도 성능이 떨어진다는 점입니다. 예전에는 그런 상황이 허용됐지만 지금은 아닙니다.

설비 적용

그래서 라이다 같은 다른 센서를 함께 쓰는 **센서 퓨전**을 시작했습니다. 연구소가 먼저 기술을 확보했고 사업실로 내려오는 중이며, 앞으로 적용이 늘어날 방향입니다.

STEP 7

취업 준비에 대하여

강사의 경로와 신입에게 기대하는 것

사전 질문에서 가장 많이 나온 주제였습니다. 강사는 본인이 면접관이 아니므로 정답은 아니라고 전제하면서 답했습니다.

역량 강점으로 문을 열고 안에서 확장한다 강사의 실제 경로

정의 하고 싶은 분야가 아니라 자신이 가장 잘 어필할 수 있는 직무로 지원하고, 입사 후에 원하는 분야로 옮겨 가는 전략입니다.

왜 쓰나 경쟁자 중에는 석사와 경력자가 많습니다. 같은 분야로 정면 경쟁하면 불리합니다. 강사는 비전을 하고 싶었지만 자신 있는 것이 코딩이었으므로 **웹 개발로 지원했습니다**. 자바 코딩테스트 성적이 좋게 나왔고 면접관도 그 점을 인지하고 있었습니다.

응용 면접에서는 지원 직무와 별개로 비전 프로젝트를 스펙으로 어필했습니다. 그 결과 웹 개발 업무를 받되 **비전 관련 안전 관제 조직에** 배정되었습니다.

개념

입사 후에는 웹 개발을 하면서 연구소 인력에게 질문을 많이 던지며 어깨너머로 배웠습니다. 비전과 AI 인력이 부족한 상황이라, 준비가 되어 있으면 회사가 먼저 **해 보겠느냐고 제안해** 옵니다.

설비 적용

대학 졸업 시점에 강사는 비전이나 빅데이터 스펙이 전혀 없었고 코딩 경험만 있었습니다. 교육을 들으며 프로젝트에서 수상해 그 격차를 메웠습니다.

역량 프로젝트와 수상 이력 교육 과정을 스펙으로 바꾸는 법

정의 교육에서 수행한 프로젝트를 결과가 남는 형태로 만들어 면접에서 말할 거리로 확보하는 것입니다.

왜 쓰나 옆자리 지원자들은 프로젝트 설명을 술술 해냅니다. 그와 겨루려면 **증명할 수 있는 것**이 필요합니다. 강사는 빅데이터와 AI 교육을 각각 들었고 두 프로젝트 모두 수상 이력을 남겼습니다.

응용 그래서 이런 문장을 만들 수 있게 됩니다. 나는 코딩만 해 왔는데 비전 AI 교육을 받으며 다른 수강생들과 경쟁해 성과를 냈다. 한 문장이지만 이력의 방향을 설명해 줍니다.

주의

다만 강사는 이 교육을 들었으니 프리패스라는 것은 아니라고 분명히 선을 그었습니다. 열심히 준비했고 결과를 남겼다면 **포스코 쪽에 관심이 있구나** 하는 어필은 된다는 정도입니다.

역량 자격증의 위치 상대적으로 판단할 것

정의 자격증은 절대 기준이 아니라 자신의 스펙 균형을 맞추는 수단이라는 관점입니다.

왜 쓰나 요즘 입사자들은 구성이 제각각입니다. 자격증이 부족해도 역량이 좋으면 뽑히는 경우가 많습니다. 다만 **역량이 평균 수준이라면 자격증도 평균은 맞춰야 합니다.**

응용 실행 방법은 자기 스펙을 냉정하게 늘어놓고 부족한 부분과 어필할 부분을 갈라 보는 것입니다. 어필할 축이 뚜렷하면 자격증에 시간을 덜 써도 되고, 그렇지 않으면 채워야 합니다.

개념 도메인 지식은 어디까지 입사 전에 다 알 필요 없다

정의 철강 공정 지식을 입사 시점에 어느 정도 갖추어야 하는가에 대한 기준입니다.

왜 쓰나 강사 본인도 아직 부족하다고 말했습니다. 담당 프로젝트에 대해 현업의 설명을 듣고 어떤 데이터와 도메인 지식이 필요한지 그때 인지하는 방식입니다. **입사할 때부터 공정 흐름을 다 아는 것을 기대하지 않습니다.**

응용 신입에게는 사내 러닝 플랫폼의 공정 강의를 먼저 듣게 합니다. 수십 시간 분량이고 들어도 잘 모르니다. 그 정도 수준의 도메인 지식이면 된다는 것이 강사의 기준입니다.

개념

그래도 조금씩 알아 두면 나중에 업무할 때 들어본 것이라는 감각이 생겨 **적응이 빨라집니다.** 미리 완벽히 아는 것이 아니라 귀에 익혀 두는 정도가 목표입니다.

역량 바이브 코딩과 코드 이해 쓰되 이해하고 쓴다

정의 생성형 AI로 코드를 만들되, 나온 코드를 반드시 이해한 상태로 쓰는 것을 말합니다.

왜 쓰나 사내에서 처음에는 개발자 몇 명에게만 코파일럿 키를 줬지만 지금은 원하는 사람 누구에게나 발급합니다. 분기나 반기마다 해커톤을 열고 그 짧은 시간의 개발도 전부 이 방식으로 합니다. **쓰는 것이 기본 전제**가 되었습니다.

응용 다만 협업에서 이 코드는 왜 이렇게 들어갔냐는 질문을 받았을 때, 그냥 AI가 그렇게 줬서 넣었다고 답하면 성립하지 않습니다. 흐름을 살펴보고 이해가 안 되는 부분은 왜 이렇게 동작하냐고 **계속 파고들면** 결국 다 이해할 수 있습니다.

주의

강사가 덧붙인 조언은 사용법에 관한 것입니다. 안 되면 붙여넣고 다시 시키는 식이 아니라, **설계를 먼저 하고 한 번의 질문으로 적은 토큰을 써서** 좋은 결과를 받는 연습을 해 보라는 것입니다.

기술 RAG 와 사내 에이전트 현직자가 실제로 쓰는 방식

정의 내부 문서를 검색해 답변의 근거로 붙이는 기법이며, 사내에서 매우 많이 쓰입니다.

왜 쓰나 공정 문서와 사내 규정은 외부 모델이 알지 못합니다. 검색으로 근거를 붙이면 폐쇄망 안에서도 문서 기반 답변이 가능해집니다. 이것만 전문으로 하는 팀이 따로 있습니다.

응용 속도가 매우 빠릅니다. 한 주 지나면 새 에이전트가 나왔으니 써 보라 하고, 몇 주 뒤에는 계획서 작성부터 특정 단계까지 순서대로 처리하는 에이전트가 나왔다는 식입니다.

치트시트

한 장으로 압축한 특강

복습할 때 이 표만 훑어도 흐름이 되살아나도록 정리했습니다.

구분	항목	핵심 한 줄
데이터	정제된 데이터	교육용은 손질되어 있고 현장은 그렇지 않다
데이터	자유도	쓸 것이 너무 많아 무엇을 쓸지부터 정해야 한다
보안	폐쇄망	공정 데이터는 외부로 나갈 수 없다
보안	버티컬 LLM	오픈소스를 받아 공정에 맞게 파인튜닝해 쓴다
비용	엣지 서버	저사양 · 저비용 · 빠른 도입, 순수 비전에 적합
비용	GPU 슬라이스	자원을 묶으로 제한, 최적화가 가능 여부를 가른다
비용	경량화 · TensorRT	같은 자원에서 처리량을 늘리는 수단
조직	연구소	POC로 가능성과 기술을 먼저 확보한다
조직	사업실	기술을 이관받아 현장에 적용하고 확산한다
현장	화각 확인	설치된 카메라를 쓸 수 있는지 직접 본다
현장	암묵지	작업자 판단은 문서에 없다, 인터뷰로 캔다
협업	개발 범위	보수적으로 긋고 추가는 고도화로 넘긴다
역량	질문과 기록	신입에게 가장 크게 기대하는 항목
운영	도커	환경을 통째로 옮겨 폐쇄망 제약을 푼다
구조	멀티 에이전트	메인이 배분하고 서브가 톨을 고른다
구조	하네스 엔지니어링	가드를 촘촘히 치고 성능이 오르면 걷어낸다
구조	PLC 레이턴시	네트워크를 타지 않게 공장 내 직결한다
지표	미탐	안전에서는 허용되지 않는다
지표	오탐	분석이 가능하므로 어느 정도 허용된다
설득	대안 제시	안 된다고 끝내지 않고 대응까지 함께 준다

오해 · 함정 사전

잘못 알기 쉬운 지점

특강 내용을 학생 입장에서 잘못 이해하기 쉬운 지점을 정리했습니다. 왼쪽이 흔한 오해, 오른쪽이 실제입니다.

X 이렇게 알기 쉽다	무슨 일이 생기나	O 바로잡기
모델 성능만 올리면 프로젝트가 된다	자원 제약과 운영 조건을 놓쳐 현장 적용에서 막힌다	성능은 기본, 최적화와 운영 설계가 함께 간다
POC가 성공했으니 본 프로젝트도 된다	원하는 화각으로 찍은 데이터라 조건이 다르다	POC는 가능성 확인, 현장 조건은 별도로 검증한다
오탐과 미탐은 같은 오류다	안전 관제에서 우선순위를 잘못 잡는다	미탐은 불가, 오탐은 분석 가능하므로 일부 허용
안 되는 것은 안 된다고 하면 된다	프로젝트 자체가 무산된다	되는 조건과 안 될 때의 대응을 함께 제시한다
도커를 능숙히 다뤄야 지원할 수 있다	준비 부담만 커지고 정작 기본기를 놓친다	리눅스와 도커를 써 본 경험이면 기본기로 충분
공정 지식을 미리 다 알아야 한다	범위가 넓어 준비가 끝나지 않는다	입사 후 사내 강의로 시작한다, 미리 귀에 익히면 유리
바이브 코딩을 쓰면 실력이 안 는다	도구를 피하다 사내 기본 방식에서 뒤처진다	쓰되 나온 코드를 끝까지 이해하는 것이 조건
가드를 많이 치면 에이전트가 아니다	에이전트 도입 필요성을 설명하지 못한다	가드는 임시, 성능이 오르면 단계적으로 걷어낸다
디지털 트윈이 곧 표준이 된다	투자 대비 효과를 과대평가한다	수십억 규모에 인력도 적어 아직 확산 단계는 아니다

하이라이트와 다음 액션

내 상황에 붙이기

C.1 특강에서 가장 밀도 높았던 세 대목

강의 시간 배분을 기준으로 강사가 가장 힘을 준 지점입니다.

대목	강조된 이유
자원 제약과 최적화	2022년 대비 가장 크게 달라진 부분이며 설명 시간이 가장 길었다
현장 인터뷰와 암묵지	만드는 것보다 쓰이게 하는 것이 어렵다는 문제의식
질문과 기록	신입에게 실제로 기대하는 유일한 항목에 가깝다

C.2 내 경험과 겹치는 지점

이번 특강 내용 중 이미 갖고 있는 경험과 직접 맞물리는 부분입니다. 자기소개서와 면접 답변으로 발전시킬 재료가 됩니다.

특강 내용	내 경험	연결 방식
비용 제약과 최적화	BISKIT POINT 오디오 청킹으로 API 비용 10분의 1 절감	자원 제약 안에서 성능을 낸 사고 경험으로 제시
현장 인터뷰와 암묵지 정리	포스텍 창업 경진대회 고객 인터뷰 14명, 3차 피봇	상충하는 진술을 모아 기준을 세운 경험으로 제시
PLC 연동과 레이턴시	임베디드 · PLC 관심, 초연결 무인체 자동시스템 대회	소프트웨어와 설비 사이를 잇는 방향성으로 제시
LLM 에이전트 구조	GPT · Whisper API 통합 및 서비스화 경험	에이전트 개발 직무와의 접점으로 제시

C.3 다음 액션

특강을 듣고 끝내지 않으려면 무엇을 할지 정해 두어야 합니다.

시점	할 일	근거
이번 주	리눅스 환경에서 도커 이미지 하나 직접 빌드해 띄워	강사가 밝힌 비전 직무 기본기 기준

시점	할 일	근거
	보기	
이번 주	아카데미 프로젝트를 문제 - 해결 - 결과 구조로 문서화	프로젝트 경험을 말할 거리로 만들라는 조언
이번 달	오픈소스 경량 모델을 로컬에서 직접 추론해 보기	폐쇄망 · 버티컬 LLM 맥락과 직결
이번 달	회의록 작성 습관을 팀 프로젝트에 적용	신입에게 기대하는 항목 1순위
졸업 전	정보처리기사 등으로 평균 수준의 자격 요건 확보	역량이 평균이면 자격증도 평균은 맞추라는 기준

C.4 특강에서 다루지 않은 것

사전 질문이 30개 이상 더 있었으나 시간상 다루지 못했다고 강사가 밝혔습니다. 아래 항목은 이번에 답을 얻지 못했으므로 다음 기회에 확인할 목록으로 남겨 둡니다.

남은 질문	왜 중요한가
전산설계 · EIC 직무의 실제 업무 비중	강사는 비전 · 에이전트 직무 기준으로만 답변했다
예지보전 과제의 라벨 확보 방식	PDM은 이번 특강 범위 밖이었다
포항 · 광양 · 판교 근무지 배정 기준	조직마다 다르다는 전제만 확인됐다
구체적 문제 해결 사례	비밀유지 계약으로 상세 공유가 어려웠다

C.5 면접에서 쓸 수 있는 문장

특강에서 확인한 기준을 그대로 활용해, 이미 가지고 있는 경험을 어떻게 말할지 미리 문장으로 만들어 둡니다. 강사가 강조한 항목과 짝을 맞췄습니다.

강사가 강조한 기준	대응하는 내 문장
제한된 자원 안에서의 최적화	대용량 오디오를 청크로 분할해 API 호출 비용을 10분의 1로 줄인 경험이 있습니다
현장 인터뷰와 기준 수립	고객 인터뷰 14명의 상충하는 진술을 정리해 세 차례 방향을 수정한 경험이 있습니다
코드를 이해하고 쓰는 것	11주간 단독으로 설계부터 배포까지 수행하며 모든 구간을 직접 다뤘습니다
운영까지 고려한 설계	CORS와 세션 구조 문제를 배포 환경에서 직접 해결했습니다

네 문장 모두 **결과 수치나 구체적 행동**을 담고 있다는 점이 중요합니다. 강사가 옆자리 지원자들은 프로젝트 설명을 술술 해냈다고 말한 대목이 바로 이 차이입니다.

오늘 얻은 것

이번 특강으로 현장에서 AI가 어떤 제약 안에서 만들어지는지에 대한 지도를 얻었습니다. 구체적으로는 다음 네 가지입니다.

첫째, 교육 데이터와 현장 데이터의 차이가 난이도가 아니라 **자유도**의 문제라는 점을 알게 되었습니다. 둘째, 보안과 비용이 기술 선택을 실제로 결정한다는 것을 폐쇄망과 GPU 슬라이스 사례로 확인했습니다. 셋째, 연구소와 사업실이라는 조직 구분이 곧 진로 선택지라는 것을 알게 되었습니다. 넷째, 신입에게 기대되는 것이 화려한 기술이 아니라 **질문과 기록**이라는 기준을 확인했습니다.

복습 루틴

이 정리본은 아래 순서로 다시 읽으면 효율이 좋습니다.

1단계로 부록 A 치트시트를 훑어 용어를 되살립니다. 2단계로 부록 B에서 잘못 알고 있던 지점이 있는지 확인합니다. 3단계로 STEP 2와 STEP 4만 다시 읽습니다. 이 두 장이 실무 감각의 핵심입니다. 4단계로 부록 C의 다음 액션 중 아직 하지 않은 것을 하나 골라 실행합니다.

연결

이후 정규 수업에서 다시 판다스와 모델링으로 돌아갑니다. 그때 이 데이터가 현장에서 왔다면 어땠을까를 한 번씩 떠올려 보면, 같은 실습에서도 훨씬 많은 것이 보입니다.